

# Cyber-Attack Attribution using Malware Artifacts

The University of Texas at Dallas

October 12, 2020

Team Ctrl-Alt-Elite

Kavya Kurup

Sydney Medina

Dhruv Mehta

Juhi Patel

Pritom Das Radheshyam

[kavya.kurup@utdallas.edu](mailto:kavya.kurup@utdallas.edu)

[sydney.medina1@utdallas.edu](mailto:sydney.medina1@utdallas.edu)

[dhruvhemant.mehta@utdallas.edu](mailto:dhruvhemant.mehta@utdallas.edu)

[juhi.patel1@utdallas.edu](mailto:juhi.patel1@utdallas.edu)

[pritomdas.radheshyam@utdallas.edu](mailto:pritomdas.radheshyam@utdallas.edu)

# Table of Contents

|                                                                                                |           |
|------------------------------------------------------------------------------------------------|-----------|
| <b>Table of Contents</b>                                                                       | <b>2</b>  |
| <b>Project Summary</b>                                                                         | <b>3</b>  |
| Description                                                                                    | 3         |
| Motivation                                                                                     | 3         |
| <b>Organization</b>                                                                            | <b>3</b>  |
| Required Skills                                                                                | 3         |
| Total Budget                                                                                   | 4         |
| Schedule                                                                                       | 4         |
| <b>Team Approach and Justification</b>                                                         | <b>5</b>  |
| <b>Technical Approach</b>                                                                      | <b>6</b>  |
| <b>Progress</b>                                                                                | <b>6</b>  |
| Data Gathering                                                                                 | 6         |
| Data Classification, Extraction, and Use                                                       | 6         |
| <b>Literature Review</b>                                                                       | <b>8</b>  |
| DeepAPT: Nation-State APT Attribution Using End-to-End Deep Neural Networks[1]                 | 8         |
| MV FCC: A Multi-View Fuzzy Consensus Clustering Model for Malware Threat Attribution[2]        | 8         |
| Applying Supervised Learning on Malware Authorship Attribution [3]                             | 9         |
| Deep learning based Sequential model for malware analysis using Windows exe API Calls[4]       | 9         |
| The Power of Threat Attribution- Kaspersky Threat Attribution Engine[5]                        | 10        |
| When Coding Style Survives Compilation: De-anonymizing Programmers from Executable Binaries[6] | 11        |
| <b>Biographical Sketches of Investigators</b>                                                  | <b>12</b> |
| Kavya Kurup                                                                                    | 12        |
| Sydney Medina                                                                                  | 12        |
| Dhruv Mehta                                                                                    | 12        |
| Juhi Patel                                                                                     | 12        |
| Pritom Das Radheshyam                                                                          | 13        |
| <b>Bibliography</b>                                                                            | <b>14</b> |

# Project Summary

## Description

Cyber attack attribution is the process of attempting to trace back a piece of code or malware to a perpetrator of a cyberattack. As cyber attacks have become more prevalent, cyber attack attribution becomes more valuable. The process of cyber attack attribution can be done using reverse engineering. From the metadata of the malware executable file, we can gather data such as date of creation, variable names used, and what library calls are imported. This information can be used as features for attribution analysis. We need to extract the features from malware that can be used for attribution and analyse them using some technique to attribute the attacks.

## Motivation

Our interest stems from the increase in recent cyber attacks. It is important to analyse the malware and get its artifacts which can give us a hint of the attacker. Cyber attacks will only become more prominent as time goes on and as our society becomes more dependent on our technological infrastructure. Cyber attack attribution will be a key element in combating malware development by helping to classify the malware by type, authorship, and other key categories. This is especially useful when you consider state-sponsored malware. It is becoming imperative to be able to identify the authorship of the malware as well as the country the malware comes from because cyber warfare will eclipse traditional combat warfare in the future. Our team has experience with machine learning algorithms and is proficient with Python.

## Organization

### Required Skills

A basic understanding of malware analysis and machine learning algorithms. Knowledge of how the tools Cuckoo and VirusTotal work and different artifacts of malware such as PE header. Lastly, effective communication with our teammates and the sponsor.

## Total Budget

There are no costs related to this project. The team will spend approximately 10 hours a week working on this project.

## Schedule

| Task / Deliverable                                                                                                       | Date              | Person In Charge           |
|--------------------------------------------------------------------------------------------------------------------------|-------------------|----------------------------|
| Bid Proposal                                                                                                             | September 4,2020  | All                        |
| Literature Review and Proposal                                                                                           | September 18,2020 | All                        |
| VM Access successful to a windows 7 image of 32GB RAM and 64GB HDD, and command prompt access of the host machine.       | October 4, 2020   | All                        |
| VirusTotal Report Generation for all the malware samples using the API.                                                  | October 7, 2020   | Kavya, Juhi, Pritom        |
| Differentiating The Malwares based on Pack or Unpacked using PEID and other tools                                        | October 11,2020   | Kavya, Dhruv, Juhi, Pritom |
| Midterm Presentation                                                                                                     | October 16,2020   | All                        |
| Midterm Report                                                                                                           | October 18,2020   | All                        |
| Extract Static Features for malwares from the “sections” and “Imports” attributes in the VirusTotal report               | October 22, 2020  | All                        |
| Build a ML model using static features as input for attribution.                                                         | October 30,2020   | All                        |
| Research on choosing any one of the approach for packed malwares:<br><br>1) Dynamic Analysis using Cuckoo Sandbox on the | November 5,2020   | All                        |

|                                                                                                                 |                             |     |
|-----------------------------------------------------------------------------------------------------------------|-----------------------------|-----|
| packed malwares<br>2) Convert packed malwares into assembly language and then further analysis for attribution. |                             |     |
| Based on the chosen method for Packed Malwares, implement the solution.                                         | November 15,2020            | All |
| Completion of any pending work and final review and optimisation of the code                                    | November 27,2020            | All |
| Project Completion and Demo                                                                                     | November 27,2020            | All |
| Final Presentation and report                                                                                   | December 4,2020 (Tentative) | All |

## Team Approach and Justification

We plan to use machine learning to attribute a nation-state sponsored malware to an APT group. The malware samples will be retrieved from GitHub repositories we find during research. We will generate reports of these malwares using the VirusTotal API. We will be using a Windows virtual machine in order to have a safe environment to download and process the malware in. We will extract various attributes from the VirusTotal reports and use them as an input to our machine learning agent to get the attribution result, which then be utilized to classify the malware into different APT groups. The justification we have for this approach comes from the research paper “Applying Supervised Learning on Malware Authorship Attribution” and other papers noted in our included Lit review. The authors of these papers classify state sponsored malware into countries and API groups by using the reports generated by various sandbox environments.

# Technical Approach

First, our team identified a GitHub repository of pre-classified state sponsored malware that we will be using as our dataset. Next, we got GUI access to a remote virtual machine with Windows 7 installed. We will use this virtual machine as our analysis environment so that there is no chance that our personal machines could get infected with any malware. We have downloaded the malware from the repository into our virtual machine. We have generated the VirusTotal reports for the malware samples. We will fetch the PE header attributes from the VirusTotal reports and use those attributes as input to our machine learning agent.

## Progress

### Data Gathering

As we mentioned in the initial project proposal, during our research we found a repository in Github that contained about 4,500 country-sponsored malware samples [3] that we decided to use as our dataset. We narrowed our scope by choosing just one APT group of malware per country, which still left us with over 2000 malware samples to classify with. We then acquired access to a virtual machine running Windows 7 that is run on UTD servers that is specifically used to test malicious code on. This environment will ensure that all of our testing will be safe and have no chance of infecting our own personal computers. We also have access to the VirusTotal API and have created a script that will send the hash value of all malware samples to the VirusTotal API and download the reports for all malwares as a json file.

### Data Classification, Extraction, and Use

Since we plan on doing static analysis, we needed to consider whether the malwares were packed or not. Packing is a form of code obfuscation that modifies the format of the malware by compressing or encrypting the data, so we would not be able to extract the artifacts we need without putting the malware through some type of unpacking software. Because of this, we used a tool called PEiD that can detect whether or not a malware sample is packed and classified all of our malware into either unpacked or packed categories. The number of packed and unpacked malwares per APT group are summarized in the table below (Fig 1). We then wrote a script that would automatically sort the malware files into packed or unpacked folders in our virtual machine environment.

| APT GROUP      | No. of unpacked files | No Information | No of packed files |
|----------------|-----------------------|----------------|--------------------|
| 1              | 353                   | 46             | 1                  |
| 10             | 466                   | 140            | 1                  |
| 19             | 474                   | 172            | 3                  |
| 21             | 57                    | 29             | 0                  |
| 28             | 68                    | 132            | 3                  |
| 29             | 71                    | 413            | 5                  |
| 30             | 193                   | 421            | 38                 |
| Dark_Hotel     | 330                   | 552            | 39                 |
| Energetic_Bear | 330                   | 680            | 40                 |
| Equation_Group | 721                   | 684            | 43                 |
| Gorgon_group   | 886                   | 759            | 147                |
| Winnti         | 1086                  | 861            | 152                |

Fig 1

We will extract the PE header attributes such as '*sections*' and '*imports*' from the VirusTotal reports and create an excel file which will be used as an input to our machine learning agent. We will create different excel sheets for packed and unpacked malware in order to see the difference in the accuracy of attribution when we perform our analysis. We may also use PeStudio tool to fetch out the strings from malware executable.

We are still in the process of researching and finalising our machine learning algorithm to classify the malware. We will use one of the following: Random Forest, K Nearest Neighbors, or Neural Networks. We are currently studying the advantages, disadvantages and the use cases of the three machine learning algorithms to best select the algorithm given our input data. Once an algorithm is chosen, we will separate our datasets (the excel sheet for the malware attributes) into training and testing data and run our algorithm on both sets to calculate our accuracy. If there are any artifacts that seem either useless or extremely important to classification, we will adjust our machine learning model accordingly.

# Literature Review

## DeepAPT: Nation-State APT Attribution Using End-to-End Deep Neural Networks<sup>[1]</sup>

The authors here describe how the nation-state APT can be attributed by using its dynamic analysis as raw input to a deep neural network (DNN) allowing it to learn high level feature abstractions of the APTs itself. The nation-state APT problem is more complex since a nation may have many cyber units with different targets, functionality and framework. Therefore, the author attribution approach will not work. APT's cannot be classified based only on frameworks and functionality. The authors used the Cuckoo Sandbox reports of samples of APT which were allegedly developed by nation-states as input to DNN. The DNN had a 10 layer fully connected network. The classifier achieved an accuracy of 94.6%. The authors concluded different nation-states use different APT developing methodologies, such that the works of developers in separate cyber units are still sufficiently similar to each other.

## MVFCC: A Multi-View Fuzzy Consensus Clustering Model for Malware Threat Attribution<sup>[2]</sup>

The authors proposed a multi-view fuzzy consensus clustering model for attributing malware. The authors used the Cuckoo sandbox reports for the malware samples and converted the report for each malware to different views in four categories namely Opcode, Bytecode, SystemCall and Header. The opcode and bytecode consists of the Binary, count, Frequency of occurrence, Term Frequency-Inverse Document Frequency and Eigen Vector modes. The authors then fuzzified each feature of all views using Trapezoidal and Triangular fuzzifier. They next used the Fuzzy Pattern tree (FPT) classification algorithm on views. FPT will result in a set of  $k$  ( $k$ = no of APT groups) classifiers being trained on each view. The authors now applied Fuzzy C-Means partitioning technique to divide malware in each APT group to several clusters. Classification and clustering provided a database of fuzzy metrics on which a decision tree was trained to provide the APT group for the malware sample. The decision tree aggregates information from different fuzzy learning algorithms. The authors were able to attribute malware samples to its APT actors correctly with 95% of accuracy. Therefore, a malicious actor needs to change all possible properties of the malware to evade or fool the MVFCC

## Applying Supervised Learning on Malware Authorship Attribution

[3]

The authors of these papers proposed a method to attribute state-sponsored authorship of malware using the supervised learning approaches Ro18 and Am17. Ro18 is a supervised learning technique that analyzes malware reports generated by Cuckoo Sandbox and inputs it into a deep neural network in order to classify state-sponsored malware. Am17 is similar to Ro18, but focuses on family classification of criminal malware instead of just state-sponsored malware, and it uses a random forest classifier instead of a DNN. The results from Ro18 when given a scenario of unseen APT groups showed that the accuracy of the classifiers was highly dependent on the sandbox reports that were used. On the other hand, when working with samples of earlier seen APT groups, the DNN works much better overall, regardless of the sandbox environment used. When using Am17, the results with unseen APT groups are worse than the DNN solution, while working with seen APT groups yields similarly good results. <https://github.com/cyber-research/APTMalware>

## Deep learning based Sequential model for malware analysis using Windows exe API Calls<sup>[4]</sup>

The authors experimented with classifying malware using a sequential model machine learning technique in a Cuckoo sandbox environment. The Long-Short Term model learning method was chosen because it allows long term dependency to be detected at random intervals. Using multiple malware datasets, they were able to test the LSTM model, but also develop data on API calls for Windows OS malware. To do so, they first started with filtering out rows that did not have 10 or more API calls. Next, they compared the hash values given by VirusTotal with the hash values of each of the malware samples to try and classify its type. Lastly, they used the joint data of the API call sequence and the malware classification to create and train a dataset. For this, decision trees, k-clustering, and support vector machines methods were used. The authors were able to test and classify malware behavior for 7107 Windows software samples. The success rate was between 83.5% and 98.5% with performance not being an issue. Although this experiment was more of a proof of concept, it did provide more insight of Windows API calls and malware correlation, as well as a dataset pertaining to this work.

## The Power of Threat Attribution- Kaspersky Threat Attribution Engine<sup>[5]</sup>

Tracking, analyzing, interpreting and mitigating constantly evolving IT security threats is a massive undertaking. Threat intelligence (TI) has true value beyond the current hype around an emerging pocket in the information security industry. TI includes a number of different services that help organizations stay protected, increase awareness and assist with investigations. When it comes to threat intelligence, threat attribution is probably the most visible process and also a point of contention.

And there is a definite reason for this. The average time from detection to response against highly sophisticated threats is usually too long due to complex investigation and reverse engineering processes. Today, in the digital era, organizations are obliged to instantly investigate, prioritize all alerts and accelerate the time to respond. Correct and timely attribution helps not only to shorten incident response times from hours to minutes but also to reduce the number of false positives. Attributing targeted attacks is a powerful and essential tool that: Evaluates if you are a target or an unintended victim, Provides insights into the actors behind the attack and their motivation, Enables effective detection, investigation, containment and response based on the knowledge of the tactics, techniques and procedures specific to the threat actor.

Some major threat attribution techniques are (1)Timestamps:One major benefit of the Portable Execution (PE) format is that compilation times are always included. Though they can be easily altered, many samples have the original timestamps. There are a few important indicators that timestamps can show. (2)Strings, debug paths and metadata: Implanted binary code can also expose interesting details about the authors. The artifacts include their preferred language as well as debug paths, which often reveal a username, as well as internal naming conventions for internal tools, projects or campaigns. Also metadata that might reveal the configuration of the developer's system. (3)Infrastructure and backend: infrastructure reuse is the most telling sign that an attacker is resurfacing or retooling. When attackers retrieve data from the email account of an exfiltration server, prepare staging or phishing servers or check on the availability of a compromised domain, they almost always use an anonymizing service. (4)Exploits: Zero-day exploits are a great source of details about the actor behind an attack. The presence of a "0-day" immediately sets an actor apart from ordinary attackers. (5)Targeted victim: Though many indicators may be faked or altered, the dynamic between attacker and victim is harder to hide or directly manipulate as it involves 'real-world' publicly known circumstances or geopolitical conflicts. This insight allows for attacker profiling. A possible outcome is the mapping of a campaign to a

geopolitical or regional situation that may point in the direction of a given perpetrating organization or nation.

## When Coding Style Survives Compilation: De-anonymizing Programmers from Executable Binaries<sup>[6]</sup>

There is a way to identify authors of a computer program based on their coding style: variable names, spacing, etc... It has been shown that source code can be attributed to authors with high accuracy, although attribution of executable binaries seem to be a lot more difficult. They used machine learning to find patterns and they decompiled the executable binary to source code. They evaluated their approach from data from Google Code Jam and had an attribution accuracy of up to 96%.

This is preserved in the compilation process and this can be extracted from the executable binary. It may be possible to infer the programmers identity if we have a set of KNOWN potential candidate programmers, along with executable binary samples known to be authored by these candidates. This has a forensics application as they can consistently identify virtual personas and individual malicious cyber operators over time. We see hackers try to obscure their binaries so it cannot be traced back to them although we find that traditional binary obfuscation, enabling compiler optimizations, or stripping debugging symbols in executable binaries results in only a modest decrease in de-anonymization accuracy. We emphasize that research challenges remain before programmer de-anonymization from executable binaries is fully ready for practical use. For example, programs may be authored by multiple programmers and may have gone through encryption.

They approach this problem using supervised machine learning, that is, we generate a classifier from training data of sample executable binaries with known authors. A drawback is that the method is inoperable if samples are not available or too short to represent authorial style.

# Biographical Sketches of Investigators

## Kavya Kurup

Kavya Kurup holds a Bachelor's degree in Software Engineering and is currently pursuing her Master's degree in Computer Science with a concentration in Information Assurance. Also, she is currently a systems engineer at T-Mobile. Through her schooling and work, Kavya has developed applications in Python and has experience with network and data analysis. Her interests lie in vulnerability research, cryptography, and network security.

## Sydney Medina

Sydney Medina is in her final semester of her undergraduate degree in computer science and is currently enrolled in fast track to pursue a masters in Information Assurance. She is currently working a Cyber Security internship at Cisco working on the Incident Response and Digital Forensics team. Last summer she interned at Lockheed Martin on the Cyber Security Blue Team. Working on these teams she has gained knowledge and familiarity with cyber security tools, and jargon, as well as knowledge of how popular cyber attacks work since she sees them being used on a daily basis during her internship.

## Dhruv Mehta

Dhruv Mehta holds a Bachelor's degree in Information Technology from India and is currently pursuing his Master's degree in Computer Science with a concentration in Information Assurance. He is a Certified Ethical Hacker (Master). Being a CEH, he has the knowledge and experience of penetration testing and its common tools. He and his team were able to find three critical vulnerabilities in a Wi-Fi Garage door camera. He is proficient in Java and Python. He has taken coursework in Machine Learning, Network Security, Information Security. Currently, he has taken the course Systems Security and Malicious Code Analysis. His interests are penetration testing, vulnerability assessment and network security.

## Juhi Patel

Juhi Patel is in her final semester of her undergraduate degree in Computer Science and is currently enrolled in fast track to pursue a masters in Information Assurance. This

past summer she has been working on designing and developing an app at UTSW that focuses on gathering data from young adults to see if they are at risk/already have depression. She has taken coursework for machine learning, database design, algorithm design, and operating systems. Currently, she is also taking a course in systems security and malicious code. She has a strong background with Java and C++ with some experience in MIPS and C.

## Pritom Das Radheshyam

Pritom Das Radheshyam is currently pursuing a Master's degree in Computer Science and has earlier graduated with a Bachelor's degree in Information Technology. He is well versed in Machine Learning, Cloud Computing and Big Data Technologies. He is an IBM certified Cloud Computing and Virtualization Expert. He has a strong background with Python and Data Visualization. His interests lie in cloud security, network security and vulnerability research.

# Bibliography

[1] Boot, Coen. *Applying Supervised Learning on Malware Authorship Attribution*. Diss. Radboud University Nijmegen, 2019.

[2] Aylin Caliskan, Fabian Yamaguchi, Edwin Dauber, Richard Harang, Konrad Rieck, Rachel Greenstadt, Arvind Narayanan “When Coding Style Survives Compilation: De-Anonymizing Programmers from Executable Binaries.” N.p., 2015. Web.  
<https://arxiv.org/abs/1512.08546>

[3] Ferhat Ozgur Catak, Ahmet Faruk Yazı, Ogerta Elezaj, Javed Ahmed “Deep Learning Based Sequential Model for Malware Analysis Using Windows Exe API Calls.” PeerJ. Computer science 6 (2020): e285–. Web.  
<https://peerj.com/articles/cs-285/>

[4] Haddadpajouh, Hamed ; Azmoodeh, Amin ; Dehghantanha, Ali ; Parizi, Reza M. “MV FCC: A Multi-View Fuzzy Consensus Clustering Model for Malware Threat Attribution.” IEEE access 8 (2020): 139188–139198. Web.

[5] Kaspersky, Eugene, et al. “The Power of Threat Attribution-Kaspersky Threat Attribution Engine”  
<https://media.kaspersky.com/en/business-security/enterprise/threat-attribution-engine-whitepaper.pdf>

[6] Rosenberg, Ishai ; Sicard, Guillaume ; David, Eli “DeepAPT: Nation-State APT Attribution Using End-to-End Deep Neural Networks.” *Artificial Neural Networks and Machine Learning – ICANN 2017 Lecture Notes in Computer Science*, 2017, pp. 91–99., doi:10.1007/978-3-319-68612-7\_11.